

Metodología de Sistemas II

Trabajo Práctico Observer

Refactoring en el mundo real - Sistema de inventario que notifica a múltiples servicios

Alumno:

Matias Carro - matiasmanuelcarro@gmail.com

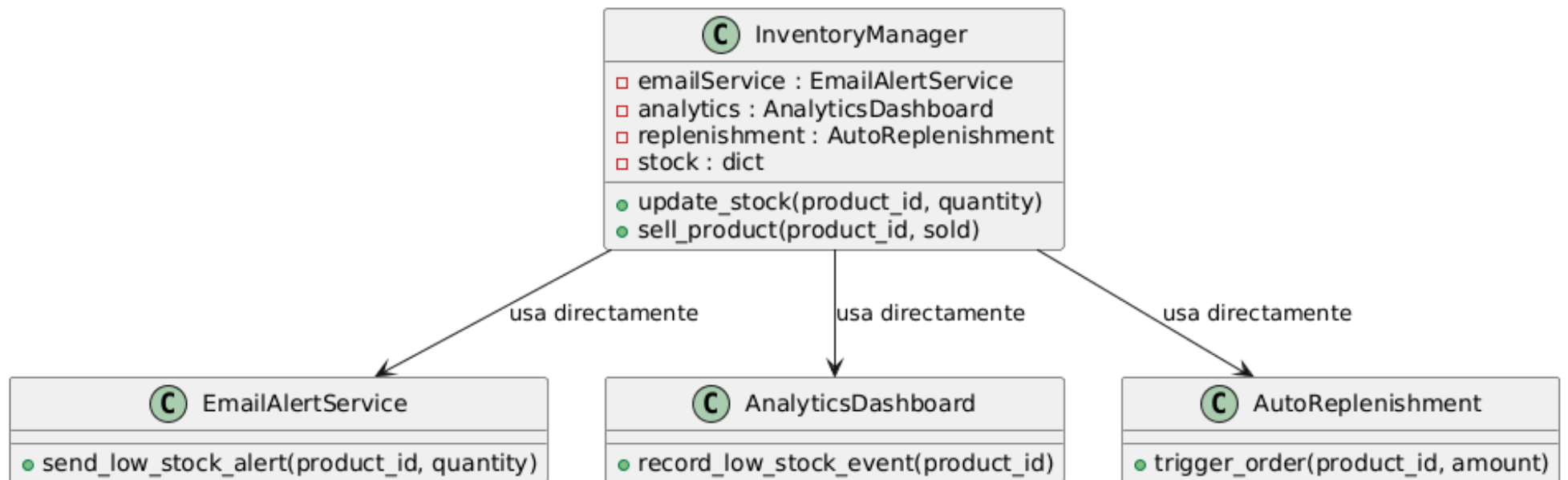
DNI: 37362003

Legajo: 18743

UML Antes.....	3
UML Después.....	4
Código antes.....	5
Código Después:.....	6
Demostración del Observer roto:.....	8
Justificación.....	8

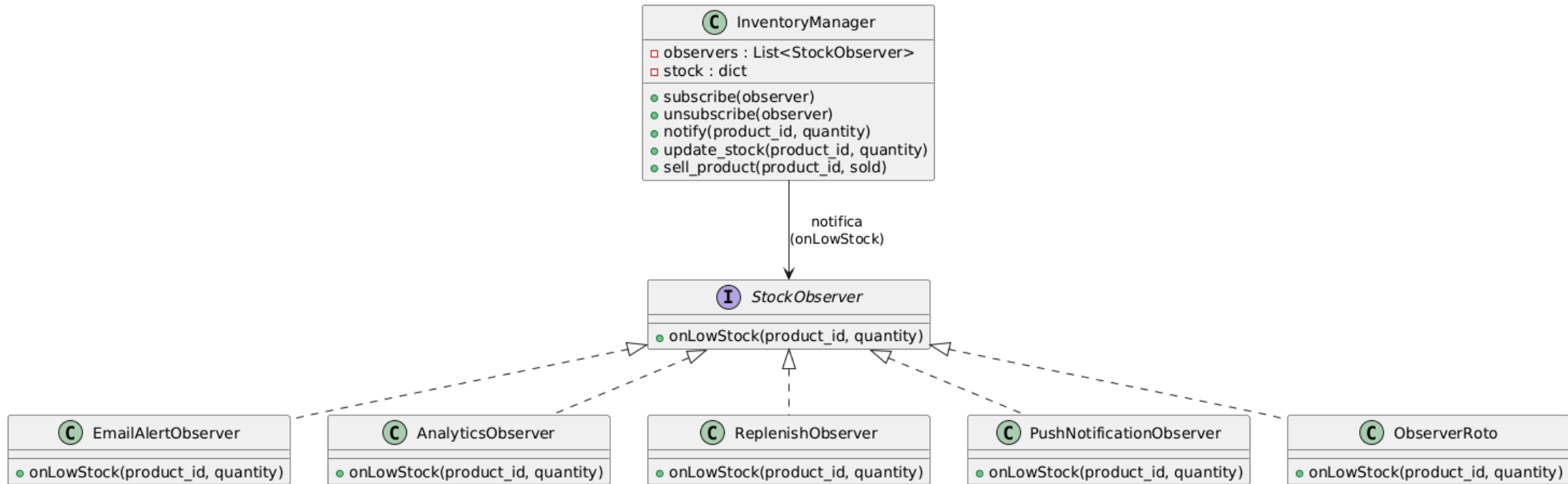
UML Antes

BEFORE - InventoryManager acoplado



UML Después

AFTER - InventoryManager con Observer



Código antes

```
class EmailAlertService:
    def send_low_stock_alert(self, product_id, quantity):
        print(f"[EMAIL] Alerta: stock bajo para {product_id} (qty={quantity})")

class AnalyticsDashboard:
    def record_low_stock_event(self, product_id):
        print(f"[ANALYTICS] Evento registrado para {product_id}")

class AutoReplenishment:
    def trigger_order(self, product_id, amount):
        print(f"[REPLENISH] Orden creada para {product_id}: {amount} unidades")

class InventoryManager:
    def __init__(self):
        self.emailService = EmailAlertService()
        self.analytics = AnalyticsDashboard()
        self.replenishment = AutoReplenishment()
        self.stock = {}

    def update_stock(self, product_id, quantity):
        self.stock[product_id] = quantity

        if quantity < 10:
            self.emailService.send_low_stock_alert(product_id, quantity)
            self.analytics.record_low_stock_event(product_id)
            self.replenishment.trigger_order(product_id, 100)

    def sell_product(self, product_id, sold):
        self.stock[product_id] -= sold

        if self.stock[product_id] < 10:
            self.emailService.send_low_stock_alert(product_id, self.stock[product_id])
            self.analytics.record_low_stock_event(product_id)
            self.replenishment.trigger_order(product_id, 100)
```

Codigo Despues:

```
# Interfaz del Observador
class StockObserver:
    def onLowStock(self, product_id, quantity):
        raise NotImplementedError("Debe implementarse en subclases")

# Excepcion propia para el sistema
class DomainError(Exception):
    """Error de dominio - Falla en observer."""

#Observadores
class EmailAlertObserver(StockObserver):
    def onLowStock(self, product_id, quantity):
        print(f"[EMAIL] Stock bajo: {product_id} (stock: {quantity})")

class AnalyticsObserver(StockObserver):
    def onLowStock(self, product_id, quantity):
        print(f"[ANALYTICS] Evento registrado para {product_id}, stock bajo con {quantity} unidades.")

class ReplenishObserver(StockObserver):
    def onLowStock(self, product_id, quantity):
        print(f"[REPLENISH] Orden creada para {product_id}: 100 unidades, stock actual: {quantity}")

# Observador nuevo para extensibilidad (notificacion push)
class PushNotificationObserver(StockObserver):
    def onLowStock(self, product_id, quantity):
        print(f"[PUSH] Notificación enviada para {product_id} (stock: {quantity})")

# InventoryManager (Publisher)
class InventoryManager:
    def __init__(self):
        self.observers = [] # lista vacía
        self.stock = {}

    def subscribe(self, observer: StockObserver):
        self.observers.append(observer)
```

```

def unsubscribe(self, observer: StockObserver):
    self.observers.remove(observer)

def notify(self, product_id, quantity):
    for observer in self.observers:
        try:
            observer.onLowStock(product_id, quantity)
        except DomainError as e:
            print(f"[ERROR] Observer falló: {e}")

def update_stock(self, product_id, quantity):
    self.stock[product_id] = quantity
    if quantity < 10:
        self.notify(product_id, quantity)

def sell_product(self, product_id, sold):
    self.stock[product_id] -= sold
    if self.stock[product_id] < 10:
        self.notify(product_id, self.stock[product_id])

# CODIGO DE PRUEBA: Observer roto
class ObserverRoto(StockObserver):
    def onLowStock(self, product_id, quantity):
        raise DomainError("error de red simulado")

# TESTS DEL SISTEMA

# Prueba de todos los observers
def test_observers():
    print("\n--- TEST: observers básicos ---")
    manager = InventoryManager()

    manager.subscribe(EmailAlertObserver())
    manager.subscribe(AnalyticsObserver())
    manager.subscribe(ReplenishObserver())

    manager.update_stock("PROD-A", 3)

# Test de falla de observer (observer roto)
def test_observer_roto():

```

```

print("\n--- TEST: observer roto ---")
manager = InventoryManager()

manager.subscribe(EmailAlertObserver())
manager.subscribe(AnalyticsObserver())
manager.subscribe(ObserverRoto())          # aca falla el observer
manager.subscribe(ReplenishObserver())     # este debe seguir funcionando

manager.update_stock("PROD-B", 5)

```

Demostración del Observer roto:

```

--- TEST: observer roto ---
[EMAIL] Stock bajo: PROD-B (stock: 5)
[ANALYTICS] Evento registrado para PROD-B, stock bajo con 5 unidades.
[ERROR] Observer falló: error de red simulado
[REPLENISH] Orden creada para PROD-B: 100 unidades, stock actual: 5

```

Se puede ver en la salida del código que el sistema continuó funcionando luego del mensaje de error. Un fallo en un observador no hace que todo el sistema frene.

Justificación

El patrón Observer resolvió el acoplamiento rígido del InventoryManager, que antes dependía directamente de tres servicios concretos. Esto hacía que cualquier cambio, falla o agregado de un nuevo servicio obligará a modificar el código del manager. También eliminó la duplicación de lógica en update_stock y sell_product. Con Observer, el manager solo conoce una interfaz, no clases concretas, y las notificaciones se vuelven extensibles sin tocar el código central. Además, cada observer maneja sus fallas sin detener a los demás, evitando que un error individual bloquee todo el flujo de notificaciones.